

DEPARTMENT OF CSE

COURSE CODE : CSE 402

COURSE TITLE : Operating System Lab

REPORT : 03

REPORT NAME : Preemptive SJF comparison with Non-Preemptive SJF

SUBMITTED BY:

NAME : SUJANA MAHMUDA NITE

STUDENT ID. : 23101120

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C-1

SUBMITTED TO:

ATIA RAHMAN ORTHI

UAP

DATE OF SUBMISSION: 03-09-2026

Problem Statement

Implement Preemptive SJF and Non-Preemptive SJF CPU scheduling for the given processes. Calculate Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT and Average WT, then compare the two approaches to determine which one performs better for the given data.

Given Data

Process	AT	BT
P1	0	4
P2	1	2
P3	3	2
P4	2	1

Objective

To study and compare Preemptive SJF (Shortest Remaining Time First) with Non-Preemptive SJF scheduling algorithms based on execution order, waiting time, turnaround time, response time, and context switching.

Working Principle

Preemptive SJF and Non-Preemptive SJF both select the shortest job first. In Non-Preemptive SJF, a process runs until completion, while in Preemptive SJF, a shorter process can interrupt the current process. Finally, their **CT, TAT, WT, and RT** are calculated and compared.

Short Comparison

Non-Preemptive SJF runs a process until it completes without interruption. In contrast, Preemptive SJF (SRTF) can interrupt the current process when a new process with a shorter remaining time arrives. Therefore, Preemptive SJF generally provides better response time and lower waiting time, but it requires more context switching.

Preemptive SJF vs Non-Preemptive SJF

Criteria	Non-Preemptive SJF	Preemptive SJF (SRTF)
Full Form	Shortest Job First	Shortest Remaining Time First
Preemption	No	Yes
Running Process	Runs until completion	Can be interrupted by a shorter process
Selection	Shortest Burst Time	Shortest Remaining Time
Response Time	Relatively higher	Generally lower
Context Switching	Less	More
Complexity	Simple	More complex
Waiting Time	Can be higher	Generally lower
Starvation	Possible	More likely

Source Code

```
# Preemptive SJF
process = ["p1", "p2", "p3", "p4"]
at = [0,1,3,2]
bt = [4,2,2,1]
n = len(process)
ct = [0] * n
wt = [0] * n
tat = [0] * n
tq= 1
remaining = bt.copy()
time = 0
completed = 0
```

```
while completed < n:
x = -1
for i in range(n):
if at[i] <= time and remaining[i] > 0:
if x == -1 or remaining[i] < remaining[x]:
x = i
if x == -1:
time += 1
else:
if remaining[x] > tq:
time += tq
remaining[x] -= tq
else:
time += remaining[x]
remaining[x] = 0
ct[x] = time
completed += 1
for i in range(n):
tat[i] = ct[i] - at[i]
wt[i] = tat[i] - bt[i]
print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
print(process[i], "\t", at[i], "\t", bt[i], "\t", ct[i], "\t", tat[i], "\t", wt[i])
p_avg_tat = sum(tat)/n
p_avg_wt = sum(wt)/n
print("\nAverage TAT =", p_avg_tat)
print("Average WT =", p_avg_wt)
```

```
print("\n")
if p_avg_tat < p_avg_tat :
print ("Preemptive TAT is best.")
else:
print("Preemptive TAT is best")
if p_avg_wt < p_avg_wt :
print ("Preemptive WT is best.")
else:
print("Preemptive WT is best")
```

Non-Preemptive SJF

```
process = ["p1", "p2", "p3", "p4"]
at = [0, 1, 3, 2]
bt = [4, 2, 2, 1]
n = len(process)
ct = [0] * n
tat = [0] * n
wt = [0] * n
done = [0] * n
time = 0
completed = 0
while completed < n:
x = -1
for i in range(n):
if at[i] <= time and done[i] == 0:
if x == -1 or bt[i] < bt[x]:
x = i
```

```
if x == -1:
time += 1
else:
time += bt[x]
ct[x] = time
done[x] = 1
completed += 1
for i in range(n):
tat[i] = ct[i] - at[i]
wt[i] = tat[i] - bt[i]
print("Process AT BT CT TAT WT")
for i in range(n):
print(process[i], at[i], bt[i], ct[i], tat[i], wt[i])
np_avg_tat = sum(tat) / n
np_avg_wt = sum(wt) / n
print("Average TAT =", np_avg_tat)
print("Average WT =", np_avg_wt)
```

Output



Discussion

In this experiment, Preemptive SJF (Shortest Remaining Time First) and Non-Preemptive SJF scheduling algorithms were studied and compared. Non-Preemptive SJF executes the selected process until it is completed, while Preemptive SJF can interrupt the current process when a new process with a shorter remaining time arrives. The values of Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), and Response Time (RT) were calculated for both algorithms. The comparison shows that Preemptive SJF can provide better response and waiting times in suitable cases, but it involves more context switching than Non-Preemptive SJF.

Conclusion

This experiment helped us understand the differences between Preemptive SJF and Non-Preemptive SJF scheduling. We learned how preemption affects process execution and scheduling performance. Overall, Preemptive SJF is useful for improving responsiveness, while Non-Preemptive SJF is simpler and requires less context switching.

GitHub Link:

https://github.com/sujana120/Operating-System-Lab